KIOSK PLATFORM · ANDROID TABLE APP

Table App Developer Guide

How to build the tablet that sits on each restaurant table: sign-in with a screen code, menu, orders, the bill, calling staff, feedback, live events and offline use. Every call here is live and covered by tests on the server.

`REST + WebSocket` `JSON · LKR cents` `en · si · ta` `OpenAPI: docs/openapi.json`

CONTENTS

1. The screens to build
2. Sign in (once, then for good)
3. Who am I
4. Menu
5. Order
6. The bill and paying
7. Calling staff and feedback
8. The visit (what the restaurant sees)
9. Live events (WebSocket)
10. Online and offline (Hybrid restaurants)
11. Staff on the tablet (optional)
12. Rules the app must keep
13. Testing against a local server

Written for: the mobile developer building the **table app**, the Android tablet that sits on each restaurant table. Guests use it to see the menu, order, call staff, ask for the bill (or pay by QR) and rate their meal. Everything below is live on the server and covered by tests.

- Base URL: the restaurant's server, e.g. `https://app.example.lk` (a Hybrid restaurant also has a local hub, see §10).
- Format: JSON over HTTPS. Times are ISO-8601 UTC.

- Money: **integer LKR cents** (`95000` = Rs 950.00); format as `Rs 950.00` .
- Names: `{"en": "...", "si": "...", "ta": "..."}` . Show the guest's language; fall back to `en` .
- Errors: always `{"error": {"code": "...", "message": "..."}}` . Show `message` ; branch on `code` .
- Full machine-readable list: `docs/openapi.json` (OpenAPI 3); a development server also shows it at `/docs` .

---

1. The screens to build

Screen	When	What it does
Sign in	No key stored, or after `401 device_not_set_up`	Restaurant address + screen code (§2)
Welcome	Idle / between guests	Restaurant logo and colour, "Tap to order", languages
Table closed	`table.open = false` (staff open each table)	Welcome + only **Call staff**
Menu	Ordering	Category pills, dish cards (photo, name, price, "+"), dish sheet with options
Cart / review	Before sending	Lines, −/+, server-priced totals (`/orders/quote`), **Send to kitchen** or **Place order**
Order status	After ordering	Placed → Cooking ("ready in about N min") → On its way → Enjoy your meal
Bill	Pay after eating	**Ask for the bill** + how: Cash / Card / QR (§6); QR shows a code
Rate	After the food is served, and after paying	5 stars, a few tags, optional comment (§7)
Call staff	Always available (a floating button)	"Priya is coming" when someone answered

Look and feel: the restaurant's own colour and logo from `venue.brand` (§3). Follow the web kiosk for the style (`frontend/src/screens/kiosk/`): white, big type, black main buttons, rectangular dish photos, no emoji.

2. Sign in (once, then for good)

The installer creates the table screen in the Setup console. It shows two things: - **Restaurant address:** e.g. `https://app.example.lk/demo`. The restaurant code is `demo`. - **Screen code:** e.g. `K7MQ-4XPA-2R`, 10 characters from `23456789ABCDEFGHJKMNPQRSTUVWXYZ`.

The console also shows a QR code holding both: `https://app.example.lk/demo?code=K7MQ-4XPA-2R`.

Step	Call
Sign in	`POST /v1/devices/sign-in` `{"venue": "demo", "code": "K7MQ4XPA2R", "client": {...}}` → `{"device_token", "device": {"id","type":"table","name","venue_code","table_ref":"T5"}, "replaced_another_device"}`
Every call after	header `X-Device-Token: <device_token>`. Keep it in the Android Keystore. It never expires
Signed out	Only **HTTP 401 with code** `device_not_set_up` means the key is gone (signed in on another tablet, code reset, or turned off). Go back to Sign in. Any other error: retry

Typing is forgiving: lower case and dashes don't matter. Errors: - `bad_screen_code`: wrong code for this restaurant. - `screen_turned_off`: the screen is turned off in the Setup console. - 429: too many tries, wait a minute.

Signing in on a second tablet signs the first one out. That keeps one table to one tablet.

`client` tells the Setup console which tablet this is. Send it with sign-in and with every heartbeat (`POST /v1/devices/heartbeat` `{"client": {...}}`, every 60 s):

```
{"install_id": "<Android ID or a UUID made once per install>", "app": "android",
 "model": "SM-X205", "os": "Android 14", "screen": "1920x1200"}
```

Start-up: with a stored key, open straight to the table screen on every start (boot, crash, update). Run as a kiosk app (lock task mode, start on boot, keep the screen on).

3. Who am I

`GET /v1/devices/me` →

```
{
  "device": {"id": "...", "type": "table", "name": "Table T5", "table_ref": "T5"}
  "venue": {"id": "...", "code": "DEMO", "name": "Demo Café Colombo", "languages"
           "dine_in_mode": "tab", "serve_mode": "waiter", "attract_images": ["/m
           "brand": {"color": "#f5a524", "logo_url": null, "theme": "light"}},
  "table": {"ref": "T5", "access": "staff_opens", "open": false},
  "payments": {"cash": true, "lankaqr": true, "card": null},
  "modules": ["R-CORE", "R-TABLE", "..."],
  "network": {"mode": "hybrid", "hub_url": "http://192.168.1.20:8000", "cloud_url
}
```


Field	What the app does
`table.access = "staff_opens"`, `open = false`	Show **Table closed**; wait for `table.state`
`venue.dine_in_mode = "tab"`	**Pay after eating:** every round goes to the kitchen at once; the main pay action is **Ask for the bill**
`venue.dine_in_mode = "pay_first"`	Guests pay each order before it is cooked (QR here, or at the counter)
`payments.lankaqr`	Offer QR payment (shown as "QR payment")
`payments.card = "reader"`	A card reader is attached: guests can tap alone. `"softpos"`: only while a staff member is signed in on the tablet. `null`: card via the waiter
`venue.brand`	Main colour (`#rrggbb`), logo (`/media/…` or null: draw a clean mark), `light` / `dark`
`modules` lacks `R-LOYAL`	Hide rewards

Reload `/v1/devices/me` on `venue.settings`.

---

4. Menu

`GET /v1/menu` → `{"categories": [{"id","name","items":[...]}], "combo_parts": {...}}`. It returns only what is served now (breakfast / lunch windows). Each item:


```
{"id","name","description","price","image_url","allergens":[...],"is_available":t
 "modifier_groups":[{"id","name","min_select","max_select","modifiers":[{"id","na
 "combo_slots":[{"name","item_ids":[...],"upcharges":{...}}], "suggest_ids":[...]
 "offer": {"price": 85500, "discount_bp": 1000, "ends_at": "..."}}
```

- **Sold out:** when `is_available = false`, show the item as Sold out and block adding it.
- **Photos:** `image_url` is relative, so prefix the server (`{server}{image_url}`). No photo: use a neutral tile.
- **Options:** respect `min_select` / `max_select` for each group, and offer the `combo_slots` choices for meals.
- **Refresh:** reload the menu on any `menu.*` event.

---

5. Order

Action	Call
Price the cart (nothing saved)	`POST /v1/orders/quote` `{"items": [line...]}` → `{"subtotal","discount_total","tax_lines":[{"code","name","rate_bp","amount"}],"tax_total","total"}`
Send it	`POST /v1/orders` `{"items": [line...]}` + header `Idempotency-Key: <uuid>`. On a network error, retry with the **same** key, never a new one
Follow it	`GET /v1/orders/{id}` + live events (§9)
Change before cooking starts	`PUT /v1/orders/{id}/items` `{"items": [...]}`
Cancel (unpaid, not started)	`POST /v1/orders/{id}/cancel` `{"reason": "..."}`

- **Cart line:** `{"menu_item_id", "quantity", "modifier_ids": [], "notes": null, "combo_choices": [], "source": null}`. Use `"source": "upsell"` when the guest added it from a suggestion.
- **Prices:** never send prices. The server prices every order.
- **Pay-after-eating (tab):** sending again adds a round to **the same order** (`is_tab: true`).
- **Order fields:** `id`, `token` (e.g. T005), `status` (placed | in_kitchen | ready | completed | cancelled | expired | voided), `payment_status` (unpaid | pending | paid | refunded), `is_tab`, `served_at`, `promised_at`, `items[{name, quantity, line_total, kds_status}]`, `subtotal`, `tax_lines`, `total`, `paid_total`, `payment_hold` (someone is taking the payment, §6).

Status to show the guest:

Condition	Show
`placed` (pay-first, not paid)	Waiting for payment
`in_kitchen`	**Cooking**, ready in about N min (from `promised_at`)
`ready` and no `served_at`	**On its way** (a waiter is bringing it)
`served_at` set	**Enjoy your meal**

6. The bill and paying

Pay after eating (the usual way): 1. The guest taps **Ask for the bill** and picks how: **Cash**, **Card** or **QR**. `POST /v1/service-requests` `{"kind": "request_bill", "order_id": "…", "pay_with": "cash"}` (`card` / `qr`). Sending again with another `pay_with` changes it, without paging twice. 2. The waiter's app shows "T5 wants the bill · Cash · Rs 2,466". The waiter prints the bill and brings it. For card, they bring the card machine. For cash, they collect it and come back with the change and the receipt. 3. Show "Your waiter is bringing the bill" (and "Priya is coming" once `coming` is set). 4. **QR:** the table can also show the code itself: `POST /v1/orders/{id}/payments/lankaqr` → `{"payment": {"id","qr_payload","amount","status":"pending"}}`. Draw `qr_payload` as a QR. It completes when the bank confirms (`order.paid`). 5. When `order.paid` arrives: "Thank you", then the rating (§7), then the welcome screen after a short idle.

Error code	Show
`being_paid`	"Your waiter is taking the payment". Someone holds the bill: `payment_hold.name`
`already_paid`	"This bill is paid. Thank you!"
`lankaqr_unavailable`	"QR payment isn't available right now. Your waiter will help"
`payment_method_off`	Hide that way to pay

Pay first (`dine_in_mode = "pay_first"`): after **Place order**, offer QR here, a card reader (`POST /v1/orders/{id}/payments/card-terminal`, completes on `order.paid`), or "Pay at the counter". Stop waiting with `POST /v1/orders/{id}/payments/{payment_id}/cancel`.

7. Calling staff and feedback

Action	Call
Call the waiter	`POST /v1/service-requests` `{"kind": "call_waiter"}`. Pressing twice doesn't page twice
This table's open calls	`GET /v1/service-requests?mine=true` → `[{"id","kind","status","pay_with","coming": {"staff_id","name"} | null}]`
Rate the meal	`POST /v1/feedback` `{"rating": 1-5, "tags": [...], "comment": "…", "order_id": "…"}` → 201. One per order; sending again changes it

- **Who is coming:** show "Priya is coming" when `coming` is set, and clear the call when it's `done`. When the bill is paid, its calls close by themselves.
- **Feedback tags:** `food`, `service`, `speed`, `value`, `clean`, `slow`, `cold`, `wrong_order`.
- **How to ask:** keep it optional — stars first, then 3–4 tag chips, then Send — and say "Thank you".
- **Low ratings:** 1–2 stars alert the staff, so a waiter comes to the table.

8. The visit (what the restaurant sees)

- **The visit:** the table is busy from the first order until the bill is paid, then it shows "Needs cleaning" until a waiter taps Cleaned.
- **Back to welcome:** on `table.cleared`, or a paid tab, forget the order and the guests' phone number and show the welcome screen.
- **Privacy reset:** after about 30 s idle with nothing open, go back to welcome. Nothing of the last guests stays on screen.

9. Live events (WebSocket)

`GET ws(s)://{server}/v1/ws?token=<device_token>` : - The first message is `{"type": "hello", "epoch", "seq"}`. - Send `{"type": "ping"}` every 25 s. - On `{"type": "resync"}`, reload the menu and your order.

Event	`data`	The app
`order.created`, `order.round_added`, `order.released`, `order.ready`, `order.served`, `order.paid`, `order.payment_hold`, `order.completed`, `order.cancelled`, `order.expired`, `order.voided`, `kds.*`	`{"order"}`	If it's your order: update the status. Chime on `order.ready`
`service.acknowledged`, `service.done`, `service.requested`	`{"request"}`	If `request.device_id` is yours: "Priya is coming" / clear
`table.state`	`{"table_ref","open"}`	Your table: show the menu, or the closed screen
`table.cleared`	`{"table_ref"}`	Guests left: back to welcome
`menu.item_availability`, `menu.updated`, `menu.offers`		Reload the menu
`venue.settings`		Reload `/v1/devices/me` (brand, payments…)
`device.command`	`{"command": "reload" | "identify"}`	Reload; or show the table's name in big letters for 8 s

Reconnect with back-off (1 s → 15 s). Close code **4401**: signed out (go to Sign in). **4409**: use the hub (§10).

---

10. Online and offline (Hybrid restaurants)

`network` in `/v1/devices/me` : - **Cloud restaurant:** talk to `server` only. - **Hybrid restaurant:** a hub box on the restaurant's Wi-Fi keeps everything working without internet. Talk to `hub_url` first and `cloud_url` when the hub can't be reached. - **Switch to the hub:** if the cloud answers **409** `use_hub` or a socket closes with **4409**. - **Back to the hub:** check `{hub_url}/v1/health` every 10 s and switch back when it answers. - **Hub address changed:** if the hub stops answering but the cloud still says `use_hub` , the hub's address changed. Read `/v1/devices/me` from the cloud again for the new `hub_url` . - **Plain http on the LAN:** the hub uses `http://` on the local network, so allow cleartext to private addresses

(`networkSecurityConfig`). Bundle the app's own screens in the APK. - **No connection at all:** show "No connection — your order is safe, we'll send it when we're back". Never send an order twice: use the same `Idempotency-Key` when retrying.

---

11. Staff on the tablet (optional)

`GET /v1/staff` lists staff names. `POST /v1/staff/pin-login` `{"staff_id","pin"}` returns `{"staff_token"}` ; send it as `Authorization: Bearer <staff_token>` too. While a staff member is signed in they can: - hold items for later (`"hold": true` on a line); - take a card tap on the tablet (SoftPOS); - order at a closed table.

Drop the staff session after 2 minutes idle.

---

12. Rules the app must keep

1. Never send prices.
2. Use an `Idempotency-Key` on every `POST /v1/orders` and every payment call.
3. Only `401 device_not_set_up` signs the tablet out. Every other error: show it, retry.
4. Call staff always works, even on the closed screen.
5. Reset to welcome after idle. Nothing personal stays on screen.
6. Card taps on the tablet only while a staff member is signed in (card-scheme rule).
7. All text in the guest's language (en / si / ta).

---

13. Testing against a local server

```
uv sync && uv run kiosk initdb && uv run kiosk seed && uv run kiosk serve --host
```

1. **Log in:** open `http://<your computer's IP>:8000/platform` with `platform@kiosk.lk` / `platform12345` .
2. **Get a screen code:** open Demo Café's setup console → Screens → add a table (T1) → note its screen code.
3. **Sign the app in:** use server `http://<IP>:8000` , restaurant `demo` and that code.

4. **Act as staff:** use the waiter app (`/demo/waiter`) and the POS (`/demo/pos`) in a browser to answer calls, take payments and so on. Staff PINs: Nimal 1111 (cashier), Priya 3333 (waiter).
5. **Fake payments (test servers only):**
6. QR paid: `POST /v1/dev/mock-gateway/{gateway_ref}/succeeded` .
7. Card tap: `POST /v1/dev/mock-terminal/{ref}/succeeded` .

Done when: - The tablet signs in once and comes back by itself after a reboot. - It shows the closed screen and opens live when staff open the table. - An order reaches the kitchen, then its status goes Cooking → On its way → Enjoy your meal. - Ask for the bill (Cash) shows on the waiter's task list, and the table resets after payment. - A rating appears in the Owner app. - Turning the hub off and on switches to the cloud and back.